Ship or not? Qwen2.5-1.5B-Instruct LoRA → merged fp16 → NF4

Executive summary

Verdict: DON'T SHIP, decided by criteria committed to the repo *before* any held-out result existed (DECISIONS.md D-20/D-21; commits `d76ee50`, `064d807`). Every number in this report traces to a raw file in `runs/`, and failed runs are kept, not deleted.

- **The model provably trained.** Its outputs moved about 10^4× further from the base than a matched null control's (lr 1e-7), the merge was verified bit for bit against an independent implementation, and the measured weight changes match what fp16 rounding predicts to four decimals.
- **The useful gain is not yet proven.** +4.0 pp accuracy (0.659 → 0.698), with a 95% paired-bootstrap CI of [−2.0, +9.9] on 252 held-out rows.
- **4-bit quantisation degrades the model in a way accuracy hides.** NF4 changes 7.9% of predictions, mostly *neutral* → *negative*.
- **Soup's built-in gates would have passed this artifact.** Given the same accuracies, `soup ship` returns SHIP. Of 21 failure modes I injected or observed across data → template → training → merge → quantisation → release, 12 passed every Soup check that applies to them; all 12 are caught by checks built for this project.

Task, setup and working conditions

Task: answer a Russian product review with a strict JSON contract, `{"тональность": "<негативный|нейтральный|позитивный>"}`. RuReviews (Apache-2.0), deduplicated and hash-split into 1500 train / 150 val / **252 held-out**, with the held-out set never used for any decision. **Training:** Soup 0.77.2 on a free Colab T4, LoRA r16/α32 on all 7 projections, on an **fp16 base**. I chose that over Soup's QLoRA default so that "merged" means exactly "what was trained" (D-10). 2 epochs, 188 steps. **Controls:** a **null control** (identical except lr 1e-7) and a **broken-template** run (Soup's `qwen2.5` alias). **Export:** `soup merge` (fp16), then Soup's `merge_4bit` (bitsandbytes NF4).

Working conditions. This was done on a secondary Windows laptop rather than my main workstation: no local GPU, about 2 GB of free RAM. I designed the workflow around that. Every script was smoke-tested end to end on CPU with Qwen2.5-0.5B before any GPU time was spent. All GPU work ran as scripted, logged Colab sessions, and raw outputs were copied back unmodified. The constraint mainly cost two follow-up experiments (end of §3); it shaped no result.

1. Memory: estimate vs measured (GiB; `analysis/estimate_vs_measured.md`)

	estimate	measured	explanation of the gap
fp16 weights / LoRA params	2.875 / 0.069	2.875 / 0.069	exact, from config.json
b4·s512·ckpt: forward / backward peak	6.40 / 8.21	6.23 / **7.33**	forward within ±3%; logits backward ≈ 11 B/element, not 14 (tensor lifetimes)
b8·s256, pre-registered "won't fit"	14.89 + ctx	14.09, **OOM at step 3**	right outcome, wrong reason: 1.34 GiB of allocator fragmentation
CUDA context / non-PyTorch	0.45 (Soup 0.8–1.2)	**0.10–0.14**	both estimators overcharge (lazy module loading)
real `soup train` run	—	allocated **6.67** · reserved **13.84** · nvidia-smi **14.3 GB**	variable batch shapes make the allocator cache twice the live tensors
GPU merge / NF4 quantise	2.93 / 1.09	3.05 / 1.10	adapter resident during merge / exact

Do I trust the reported VRAM? Only after asking which number it is: nvidia-smi showed 93% full for a run that needs about 7 GiB. **Tok/s?** Soup reports 0.26 it/s; the data processed 700 tok/s, but only about 74 *supervised* tok/s.

2. Proof of training: base vs merged vs 4-bit (`scripts/verify.py`; output `runs/phase5/verify_*/report.md`)

Why a falling loss proves nothing here. The base already produces valid JSON on 100% of rows. 13 of the 16–18 supervised tokens per row are identical across all rows, so effectively one token carries the decision. The null control's eval loss also fell (0.754 → 0.728) while its predictions stayed identical to the base.

The check. It runs on the same 252 prompts in four layers:

- weight deltas, scored against the fp16 rounding floor;
- an adapter-scale ablation (0 / 0.5 / 1);
- teacher-forced KL at the decision token, compared with a quantisation reference;
- greedy behaviour with paired bootstrap CIs.

Its own sanity conditions (T0) are checked first, and all of them hold.

	real	null control	broken template
T0: scale 0 = base · merged = adapter-on (KL)	0 · 1.2e-5 ✓	0 · 2.2e-5 ✓	0 · 1.2e-5 ✓
adapted fp16 weights changed (predicted)	**93.6%** (93.6%)	**5.9%** (5.9%)	94.2%
decision-token KL(base‖merged)	**0.370**	**3.4e-5**	0.433
Δ accuracy merged − base [95% CI]	+3.6 pp [−2.4, +9.5]	0.0 [0, 0]	+4.0 pp [−2.0, +10.3]

Soup's merge equals our independent peft merge bit for bit, and all 142 untouched tensors are unchanged. **Not detected:** wrong gold labels, generalisation beyond this prompt, damage to other abilities, other runtimes (GGUF/vLLM), free-running divergence, effects under about ±6 pp, seed variance.

3. Silent failures (fault injection; full 21-row table: `analysis/silent_failures.md`)

Each fault was injected on purpose and run through Soup's built-in checks and through ours. ✅ flagged · ❌ passed silently.

fault	data doctor	doctor / plan / dry-run	soup ship	ours
held-out rows leaked into train	❌	❌	❌	✅ hash check
max_length truncates every answer	✅ MAJOR	❌ "Ready to train"	—	✅
`chat_template: qwen2.5` drops the system prompt	❌	❌	❌ SHIP	✅ (T4: −63k tokens)
template override never trains EOS / system prompt only in train data	❌	❌	—	✅
fp16 eager attention: NaN, every step skipped (T4)	—	—	—	✅ telemetry
`lora_alpha` edited after training; bf16 weight → `inf` in fp16 merge	—	merge exit 0, no warning	—	✅
NF4 flips 7.9% of labels at equal accuracy (T4)	—	—	❌ SHIP	✅
one extra correct row of 252	—	—	❌ SHIP	✅ CI gate

What `soup ship` evaluates (read from source, and its rule run with our numbers):

- base + adapter in bf16, never the merged or 4-bit file unless it's passed explicitly via `--tuned`;
- decision = `tuned > base` on a point estimate, with no confidence interval.

Proposed Soup changes:

- a CI-based ship gate that evaluates the exported artifact;
- `--dry-run` that fails on data-doctor MAJOR findings, with data doctor seeing the template override;
- `soup merge` warnings for template overrides and for non-finite values after the dtype cast.

Not run (needs a T4 session): a live 1.5B `soup ship` run, and an NF4 ablation (no double-quant, FP4, 8-bit). Both are scripted and ready (`notebooks/colab_runner.ipynb`, Phase 6), and neither can change the verdict.

4. Verdict against the pre-registered criteria (`analysis/verdict.md`)

S1 trained & separated	S2 ≥ +5 pp, CI > 0	S3 4-bit fidelity	S4 format/EOS	S5 no collapse	S6 integrity	S7 user sampling
FAIL	**FAIL** +4.0 [−2.0, +9.9]	**FAIL** agreement 92.1%	PASS	PASS	PASS	PASS

DON'T SHIP. What must change first:

1. Re-test on ≥1,500 of the 6,863 never-used held-out rows, with thresholds registered first.
2. Fix the 4-bit step until it agrees with fp16 on ≥95% of labels.
3. Audit the neutral-class labels, which are star-derived.

A post-hoc note, clearly separated from the verdict: the pre-registered quantisation reference (T2) turned out to measure a systematic shift, not noise. Measured against the null control the real run passes T2 easily, but S2 and S3 still fail, so the verdict doesn't depend on that choice.

5. What surprised me, and what still concerns me

What surprised me most was how many wrong things look right. The null control's loss fell and Soup printed "0.776 → 0.693" for a model whose predictions never changed. The broken template matched the real run's accuracy (0.698); only a 63,000-token gap in the telemetry and a template diff exposed it. 4-bit quantisation wasn't noise but a directional shift: NF4 almost deletes the base model's *neutral* class (recall 0.57 → 0.07), and on the trained model it moved 7.9% of labels while accuracy stayed flat, because the errors cancel. What still concerns me: the verdict rests on 252 star-derived labels and one seed, and my quantisation reference was a poor choice of yardstick. My own checks have blind spots too: my label inspector missed 2 of 9 injected faults until I extended it, my eval would mask a missing EOS in `generation_config`, and an edited `lora_alpha` passes my verification. `soup ship`, given these same numbers, would have shipped this model.